EFES Final Project Report

FPGA-Assisted ECG Monitor with real-time QRS detection

Asma Rastgou
Politecnico di Torino

March 11, 2026

Contents

1 System Objective and End-to-End Architecture **3**

2 Analog Front-End, Signal Levels, and Power Integrity **4**
2.1 AD8232 Analog Front-End 4
2.2 Why an Extra RC Low-Pass Was Not Added at the ADC Input 4
2.3 ADC Input Range and Quantization 5
2.4 Power Integrity and Bypass Capacitors 5
2.5 Actuator Stage: LED, Buzzer, Transistor, and Diode 6

3 Microcontroller Acquisition Chain **7**
3.1 Clocking and Sampling Schedule 7
3.2 ADS1115 Configuration 7
3.3 I^2C Bus and Pull-Up Resistors 8
3.4 IIR DC-Bias Removal in the MCU 8

4 MCU-to-FPGA Transport and Clock-Domain Crossing **10**
4.1 GPIO-Based SPI Sample Link 10
4.2 Why No FIFO Was Needed in the FPGA Input Stage 10

5 FPGA Processing, QRS Detection, and Alarm Generation **11**
5.1 Operating Clock and Reset Strategy 11
5.2 FIR Filter Design 11
5.3 QRS Detection and BPM Estimation 12
5.4 Alarm Comparator and Buzzer Tone 13

6 Flash Logging and Storage Margin **14**

7 Physical Interconnection and Build Strategy **15**
7.1 Wiring from Sensor to Actuator 15
7.2 Breadboard Layout Rules 15

8 System Boot, Validation, and Remaining Work **16**
8.1 Boot Sequence 16
8.2 Validation Strategy 17
8.3 Limitations and Future Extensions 17

1 System Objective and End-to-End Architecture

This project was developed as a complete mixed-signal ECG monitoring chain rather than as an isolated analog or digital exercise. The objective was to start from the electrode signal, condition it, digitize it, transfer it deterministically into an FPGA, detect QRS activity in real time, log filtered data into flash memory, and finally drive an actuator when the detected heart rate falls outside the chosen alarm band.

The end-to-end signal path implemented in the demonstrator is

$$\text{electrodes} \rightarrow \text{AD8232} \rightarrow \text{ADS1115} \rightarrow \text{XMC2Go} \rightarrow \text{Tang Nano 9K} \rightarrow \text{LED + passive buzzer.}$$

I chose a sampling rate of 250 samples/s because it is high enough for ECG monitoring while still making the rest of the system easy to analyze mathematically and implement on modest hardware. The corresponding sample period is

$$T_s = \frac{1}{f_s} = \frac{1}{250} = 4\,\text{ms}.$$

Since each sample is represented as a 16-bit word, the sustained raw sample payload is only

$$R_{\text{sample}} = f_s \cdot N_{\text{bits}} = 250 \cdot 16 = 4000\,\text{bit/s}.$$

That value shaped many later design choices. A 4 kbit/s signal stream does not require a complex transport, does not justify a deep FIFO between the MCU and the FPGA, and leaves a very large computational margin inside a 27 MHz FPGA clock domain. The real challenge of the project was therefore not throughput, but signal integrity, timing determinism, and hardware integration.

2 Analog Front-End, Signal Levels, and Power Integrity

2.1 AD8232 Analog Front-End

The ECG electrodes are connected to the AD8232 module through the usual `RA`, `LA`, and `RL` inputs. I chose a commercial AD8232 breakout because it already contains the instrumentation front-end, lead biasing, and the analog conditioning blocks needed for single-supply ECG acquisition. This allowed the project effort to focus on system integration and signal processing rather than on designing a discrete ECG amplifier from scratch.

The key point is that the AD8232 does not deliver a waveform centered around 0 V. When powered from a 3.3 V single supply, its output is biased close to mid-rail. The signal may therefore be described as

$$v_{\mathrm{AD8232}}(t) = V_{\mathrm{CM}} + v_{\mathrm{ECG,ac}}(t), \qquad V_{\mathrm{CM}} \approx \frac{V_{DD}}{2} = \frac{3.3}{2} \approx 1.65\,\mathrm{V}.$$

This bias is not a defect; it is what makes single-ended acquisition possible. A waveform centered around 0 V would immediately violate the ADC input range whenever the ECG swings negative. By shifting the signal to approximately 1.65 V, the AD8232 makes it compatible with the 0 to 3.3 V analog chain.

2.2 Why an Extra RC Low-Pass Was Not Added at the ADC Input

Many ADC application notes recommend a first-order RC low-pass in front of the converter input. The cutoff frequency of such a network is

$$f_c = \frac{1}{2\pi RC}.$$

That recommendation must be interpreted carefully in this project, because the AD8232 module already performs analog filtering internally, and the ADC documentation also warns against making the series resistance too large. If one respects the conservative bound

$$R \leq 1\,\mathrm{k}\Omega$$

and tries to place the external RC cutoff near the upper edge of the ECG band, say around

$$f_c \approx 50\,\mathrm{Hz},$$

the required capacitor becomes

$$C = \frac{1}{2\pi R f_c} = \frac{1}{2\pi \cdot 1000 \cdot 50} \approx 3.18\,\mu\mathrm{F}.$$

That is already a large capacitor for a sensitive analog node on a breadboard, and it would be added on top of the filter poles already embedded in the AD8232. For this reason, the choice was made not to insert a second explicit RC stage between the AD8232 output and `AIN0`. The remaining cleanup is instead split between the MCU baseline-removal stage and the FPGA FIR filter, both of which are easier to tune than a large capacitor on a breadboard analog node.

2.3 ADC Input Range and Quantization

The ADS1115 was configured for the $\pm 4.096\,\mathrm{V}$ range even though the actual analog chain is physically bounded by the 0 to 3.3 V supply. This was a deliberate robustness choice: I preferred to lose some code-range utilization rather than risk clipping during early hardware bring-up.

The nominal ADS1115 LSB value in this configuration is

$$\Delta V_{\mathrm{LSB}} = \frac{4.096\,\mathrm{V}}{32768} = 125\,\mu\mathrm{V/LSB}.$$

With a mid-rail signal centered near 1.65 V, the corresponding nominal code is

$$\mathrm{Code}_{\mathrm{midrail}} \approx \frac{1.65\,\mathrm{V}}{125\,\mu\mathrm{V}} \approx 13200.$$

Even the physical rail of 3.3 V corresponds only to

$$\mathrm{Code}_{3.3\mathrm{V}} \approx \frac{3.3\,\mathrm{V}}{125\,\mu\mathrm{V}} \approx 26400,$$

which is still below the signed positive limit of 32767. The practical meaning is simple: the chosen ADC range is conservative, but safe.

2.4 Power Integrity and Bypass Capacitors

The breadboard build required a clean 3.3 V rail and a single shared ground reference. This was especially important because the chain mixes a high-gain analog front-end, digital switching activity, and an actuator stage.

Two classes of bypass capacitor were used. First, a local ceramic capacitor of 100 nF was placed close to each sensitive supply pin. The impedance of a capacitor is

$$|X_C| = \frac{1}{2\pi f C}.$$

At 1 MHz, a 100 nF capacitor presents approximately

$$|X_C| = \frac{1}{2\pi \cdot 10^6 \cdot 100 \cdot 10^{-9}} \approx 1.59\,\Omega,$$

which makes it effective for shunting relatively high-frequency supply noise.

Second, a bulk capacitor in the $10\,\mu$F to $47\,\mu$F range was added to the local rail. Its purpose is better described by the droop relation

$$\Delta V = \frac{I\,\Delta t}{C}.$$

For example, if a 10 mA transient lasts 100 μs, then a 10 μF capacitor limits the idealized droop to

$$\Delta V = \frac{10\,\text{mA} \cdot 100\,\mu\text{s}}{10\,\mu\text{F}} = 0.1\,\text{V},$$

whereas a 47 μF capacitor reduces the same droop to roughly 21 mV. The ceramic and bulk capacitors therefore serve different purposes and are not interchangeable.

2.5 Actuator Stage: LED, Buzzer, Transistor, and Diode

The final alarm output is generated in the FPGA and then applied to an NPN transistor used as a low-side switch. I chose this topology because the FPGA pin should provide a logic-level control signal, not the current required by the buzzer branch itself.

With a 3.3 V logic high, an approximate base-emitter drop of 0.7 V, and a 1 kΩ base resistor, the transistor base current is

$$I_B \approx \frac{V_{\text{FPGA}} - V_{BE}}{R_B} = \frac{3.3 - 0.7}{1000} \approx 2.6\,\text{mA}.$$

This is more than sufficient to drive a small transistor such as `2N2222`, `PN2222A`, or `BC547` into saturation for the modest current of the alarm stage. A 10 kΩ pull-down resistor was added from base to ground so that the transistor remains off while the FPGA is configuring.

For the LED branch, the resistor was chosen from

$$I_{LED} \approx \frac{V_{DD} - V_F - V_{CE(sat)}}{R_{LED}}.$$

Assuming $V_{DD} = 3.3\,\text{V}$, $V_F \approx 2.0\,\text{V}$, and $V_{CE(sat)} \approx 0.2\,\text{V}$, one obtains

$$I_{LED}(330\,\Omega) \approx \frac{3.3 - 2.0 - 0.2}{330} \approx 3.3\,\text{mA},$$

$$I_{LED}(1\,\text{k}\Omega) \approx \frac{3.3 - 2.0 - 0.2}{1000} \approx 1.1\,\text{mA}.$$

That is why 330 Ω was suggested when a bright indicator is desired, while 680 Ω to 1 kΩ is better when lower current is preferred.

The buzzer positive terminal is tied to the supply rail and its negative terminal is switched by the transistor collector. A flyback diode was added across the buzzer because the selected part behaves as an inductive load in practice. The diode is connected with its cathode to the positive rail and its anode to the collector node so that it remains reverse-biased during normal operation and conducts only when the magnetic field collapses.

3 Microcontroller Acquisition Chain

3.1 Clocking and Sampling Schedule

The MCU firmware assumes a CPU clock of 32 MHz and a peripheral clock of 64 MHz. The global acquisition heartbeat is generated by the Cortex-M0 SysTick timer. Since SysTick uses the core clock in this design, the reload value is

$$\text{RELOAD} = \frac{f_{\text{CPU}}}{f_s} - 1 = \frac{32\,000\,000}{250} - 1 = 127999.$$

This produces the 4 ms sampling slot used by the rest of the system.

The choice of 250 samples/s is convenient mathematically because it matches an ADS1115 data-rate option exactly. The conversion time is therefore

$$T_{\text{conv}} = \frac{1}{250} = 4\,\text{ms},$$

which means one new conversion can be made ready for each scheduler tick.

3.2 ADS1115 Configuration

The ADS1115 address was fixed at `0x48` by tying `ADDR` to ground. The converter is used in single-shot mode on `AIN0/GND`, so each measurement cycle follows the sequence

$$\text{write config} \rightarrow \text{poll OS bit} \rightarrow \text{read conversion register}.$$

The chosen configuration word is `0xC3A3`, or

$$\texttt{0xC3A3} = \texttt{1100 0011 1010 0011}_2.$$

The important fields are:

- `OS = 1`: start one conversion,
- `MUX = 100`: measure `AIN0` with respect to ground,
- `PGA = 001`: select the ± 4.096 V range,
- `MODE = 1`: single-shot mode,
- `DR = 101`: select 250 samples/s.

Single-shot mode was preferred because it makes the MCU the explicit owner of the time base: the firmware decides when the ADC converts, when the result is harvested, and when the result is sent onward.

3.3 I^{2}C Bus and Pull-Up Resistors

The ADS1115 is connected to the XMC2Go through I^{2}C on `P2.10` for `SDA` and `P2.11` for `SCL`. The firmware targets 100 kHz Standard-mode I^{2}C. One bit on that bus occupies

$$T_{\text{bit}} = \frac{1}{100\,000} = 10\,\mu\text{s},$$

and one byte plus ACK occupies roughly

$$T_{\text{byte+ACK}} \approx 9T_{\text{bit}} \approx 90\,\mu\text{s}.$$

This is comfortably below the 4 ms ADC conversion interval, so protocol overhead remains small.

The pull-up resistors were chosen as 4.7 kΩ. Their effect can be estimated from the approximate rise-time relation

$$t_r \approx 0.8473 R_P C_{\text{bus}}.$$

If the local breadboard bus is approximated as 100 pF, then

$$t_r \approx 0.8473 \cdot 4.7\,\text{k}\Omega \cdot 100\,\text{pF} \approx 0.40\,\mu\text{s}.$$

That remains below the 1 μs class typically associated with Standard-mode I^{2}C, which is why 4.7 kΩ is a reasonable default for this short local link.

3.4 IIR DC-Bias Removal in the MCU

The ADC must sample the AD8232 output with its DC bias still present, but the FPGA detector works much better if the incoming digital stream is centered around zero. For this reason, the MCU implements a first-order recursive baseline estimator, which is the DC-bias-removal IIR stage used in the project:

$$b[n] = b[n-1] + \alpha\big(x[n] - b[n-1]\big), \qquad y[n] = x[n] - b[n].$$

In firmware,

$$\alpha = \frac{1}{128},$$

which is implemented efficiently as a right shift rather than a division.

The first equation is a first-order low-pass IIR that tracks only the slow baseline component. Subtracting that estimate from the raw sample yields a first-order high-pass response. In the z-domain, the equivalent transfer function from input $x[n]$ to corrected output $y[n]$ is

$$H(z) = \frac{Y(z)}{X(z)} = \frac{(1-\alpha)(1-z^{-1})}{1-(1-\alpha)z^{-1}}.$$

This makes the pole location explicit:

$$z_p = 1 - \alpha = 1 - \frac{1}{128} = \frac{127}{128},$$

which shows that the filter is intentionally very slow and therefore acts mainly on DC offset and baseline wander rather than on the QRS content itself.

For a slow first-order tracker with small α, the equivalent analog corner can be approximated by

$$f_c \approx \frac{\alpha f_s}{2\pi}.$$

Substituting $\alpha = 1/128$ and $f_s = 250\,\text{Hz}$ gives

$$f_c \approx \frac{(1/128) \cdot 250}{2\pi} \approx 0.31\,\text{Hz}.$$

This is slow enough to follow baseline wander but fast enough to center the digital stream before it reaches the FPGA.

4 MCU-to-FPGA Transport and Clock-Domain Crossing

4.1 GPIO-Based SPI Sample Link

The corrected samples are forwarded from the MCU to the FPGA through a GPIO-generated SPI Mode 0 link. The physical mapping is

$$\texttt{P0.0} \rightarrow \texttt{SPI_CS_N_IN}, \qquad \texttt{P0.7} \rightarrow \texttt{SPI_MOSI_IN}, \qquad \texttt{P0.8} \rightarrow \texttt{SPI_SCLK_IN}.$$

I deliberately kept this link simple because the sustained payload is only 4 kbit/s. For a first hardware build, a slow and observable software SPI link is often more valuable than an aggressive peripheral configuration that is harder to debug.

The firmware bit delay is created by a short software loop of 16 iterations. One serial bit uses three such delay windows, so the bit period may be described as

$$t_{\text{bit}} \approx 3N_{\text{delay}}t_{\text{loop}} + t_{\text{GPIO}}, \qquad N_{\text{delay}} = 16.$$

The exact value of t_{loop} depends on compiler output and instruction timing, so the link was intentionally treated as a conservative bring-up interface rather than as a calibrated SPI clock source.

4.2 Why No FIFO Was Needed in the FPGA Input Stage

The FPGA receives one 16-bit sample every 4 ms, while its internal logic runs at 27 MHz. The number of core cycles available between samples is therefore

$$N_{\text{cycles/sample}} = \frac{27\,000\,000}{250} = 108000.$$

That enormous timing margin explains why the SPI receiver was implemented with a single holding register and a toggle-based clock-domain crossing rather than a deeper FIFO. The new sample simply arrives far too slowly for buffering to become a throughput bottleneck.

5 FPGA Processing, QRS Detection, and Alarm Generation

5.1 Operating Clock and Reset Strategy

The Tang Nano 9K uses its on-board 27 MHz oscillator as the main system clock. I chose to keep this operating clock because it is already present on the board, is easy to constrain, and is much faster than the sample rate required by the application. The design is therefore intentionally conservative rather than overclocked.

Reset assertion is asynchronous, but release is synchronized with a two-stage register chain. This is standard practice in synchronous digital systems: reset should assert immediately, but its release should be aligned with the clock domain that consumes it.

5.2 FIR Filter Design

The main low-pass filter in the FPGA is a 31-tap fixed-point FIR filter:

$$y[n] = \sum_{k=0}^{30} b_k x[n-k].$$

I chose 31 taps because this is long enough to produce useful attenuation above the ECG band while still remaining lightweight enough for a sequential multiply-accumulate implementation.

The intended passband is approximately below 50 Hz. Since the sample rate is 250 Hz, the Nyquist frequency is

$$f_N = \frac{f_s}{2} = 125\,\text{Hz},$$

and the normalized cutoff is therefore

$$\frac{f_c}{f_N} = \frac{50}{125} = 0.4.$$

Because the filter is linear-phase and has odd length, its group delay is

$$\tau_g = \frac{N-1}{2f_s} = \frac{31-1}{2 \cdot 250} = 60\,\text{ms}.$$

This amount of latency is entirely acceptable in a monitor-and-alarm pipeline.

The filter is implemented as a sequential MAC engine rather than as a fully parallel structure. The compute time per output sample is approximately

$$T_{\text{MAC}} \approx \frac{31}{27\,000\,000} \approx 1.15\,\mu\text{s},$$

which is negligible compared with the 4 ms sample interval. Quantitatively, this is why the sequential architecture is justified.

5.3 QRS Detection and BPM Estimation

After filtering, the FPGA detects QRS events by comparing the absolute value of the filtered waveform against a fixed threshold:

$$Q_{\text{th}} = 500.$$

This number should not be presented as a medically derived or final detector threshold. In the current prototype it is only a provisional bring-up value in the *filtered fixed-point domain*, lowered so that QRS-like events would be visible during hardware testing and debug with the present signal scaling. It is not a direct physical millivolt threshold, because both the AD8232 analog gain and the FIR scaling affect the final numeric amplitude, and it was not extracted from a calibrated dataset.

In a more mature version of the project, the threshold should be chosen from measured filtered-waveform statistics rather than fixed by hand. A generic way to express that future refinement is

$$Q_{\text{th,final}} = \mu_{|y|} + k\sigma_{|y|}$$

or, alternatively, as a fraction of a running peak estimate. The current value $Q_{\text{th}} = 500$ should therefore be read only as a practical testing constant.

The detector measures the R-R interval in samples and converts it into BPM through

$$\text{BPM} = \frac{60}{T_{RR}} = \frac{60 f_s}{N_{RR}}.$$

With $f_s = 250\,\text{Hz}$,

$$\text{BPM} = \frac{15000}{N_{RR}}.$$

The current RTL accepts the physiological range 20 to 240 BPM. Converting those limits into sample counts gives

$$N_{RR,\min} = \left\lceil \frac{15000}{240} \right\rceil = 63, \qquad N_{RR,\max} = \frac{15000}{20} = 750.$$

Intervals outside that range are rejected.

The refractory interval is set to 120 ms, which becomes

$$N_{\text{refractory}} = f_s \cdot 0.12 = 250 \cdot 0.12 = 30$$

samples. This suppresses double-counting of wide QRS complexes and the following T-wave. A simple hysteresis is also present, because the detector clears its internal `above` flag only when the filtered magnitude falls below half threshold:

$$Q_{\text{clear}} = \frac{Q_{\text{th}}}{2} = 250.$$

5.4 Alarm Comparator and Buzzer Tone

In the final FPGA alarm stage, the alarm becomes active whenever the estimated heart rate leaves the interval

$$50 \leq \mathrm{BPM} \leq 100.$$

The final actuator path is therefore FPGA-driven; the older MCU-side alarm placeholder is not the primary hardware alarm mechanism of the demonstrator.

When the alarm is active, the FPGA generates a square-wave tone rather than a static logic level. The divider constant is

$$N_{\mathrm{half}} = 6750,$$

so the buzzer frequency is

$$f_{\mathrm{buzzer}} = \frac{f_{\mathrm{FPGA}}}{2N_{\mathrm{half}}} = \frac{27\,000\,000}{2 \cdot 6750} = 2000\,\mathrm{Hz}.$$

I selected 2 kHz because it is readily audible on a small passive buzzer and easy to generate digitally from the available clock.

6 Flash Logging and Storage Margin

The external W25Q64FV flash is written one page at a time. Since one page contains 256 bytes and each filtered sample is stored as a 16-bit word, the number of samples that fit in one page is

$$N_{\text{samples/page}} = \frac{256}{2} = 128.$$

At the project sample rate, the time required to fill one page is

$$T_{\text{page fill}} = \frac{128}{250} = 0.512\,\text{s}.$$

The flash byte shifter uses `CLK_DIV` $= 4$. Since the serial clock toggles every `CLK_DIV` FPGA cycles, the flash SPI clock is

$$f_{\text{SCK,flash}} = \frac{27\,\text{MHz}}{2 \cdot 4} = 3.375\,\text{MHz}.$$

The 256-byte payload alone therefore requires

$$N_{\text{bits}} = 256 \cdot 8 = 2048\,\text{bits},$$

$$T_{\text{payload shift}} = \frac{2048}{3.375 \times 10^6} \approx 607\,\mu\text{s}.$$

Even after adding the page-program header and the flash internal write time, the logger still operates far below its throughput ceiling. If one budgets approximately

$$T_{\text{page total}} \approx 0.607\,\text{ms} + 0.75\,\text{ms} \approx 1.36\,\text{ms},$$

then the margin relative to the 512 ms page-fill interval is still

$$\text{margin ratio} \approx \frac{512\,\text{ms}}{1.36\,\text{ms}} \approx 376.$$

This quantitative margin is the reason page-based logging is safe in the present architecture.

The logger starts at

$$\texttt{START_ADDR} = \texttt{0x100000} = 1\,048\,576\,\text{bytes} = 1\,\text{MiB},$$

which intentionally leaves the low flash region untouched during early experiments.

7 Physical Interconnection and Build Strategy

7.1 Wiring from Sensor to Actuator

The physical prototype required care in both voltage-domain management and physical layout. All modules in the active chain operate in the 3.3 V domain and all grounds are tied together. This common reference is essential because the analog front-end, the ADC, the MCU, the FPGA, and the actuator stage all exchange information through signals that are meaningful only when their references match.

The physical wiring is summarized below:

- **Electrodes to AD8232:** connect the ECG electrodes to `RA`, `LA`, and `RL`.
- **AD8232 supply:** connect `3.3V` and GND; tie `SDN` high if present.
- **AD8232 output to ADC:** connect `OUTPUT` to `ADS1115 AIN0`.
- **ADS1115 static pins:** connect `VDD` to 3.3 V, `GND` to ground, `ADDR` to ground, and leave `ALERT/RDY` unused.
- **ADS1115 to MCU:** connect `SDA` to `P2.10` and `SCL` to `P2.11`, with 4.7 kΩ pull-ups to 3.3 V if they are not already present on the board.
- **MCU to FPGA:** connect `P0.0` to `SPI_CS_N_IN`, `P0.7` to `SPI_MOSI_IN`, and `P0.8` to `SPI_SCLK_IN`.
- **FPGA alarm output:** use `ALARM_OUT` on package pin 30 as the final alarm source.
- **Alarm transistor stage:** connect FPGA pin 30 to the transistor base through 1 kΩ, add a 10 kΩ base pull-down, tie emitter to ground, and tie collector to the low side of the buzzer/LED branch.
- **Buzzer branch:** connect buzzer positive to the supply rail, buzzer negative to the collector node, and place the flyback diode across the buzzer.
- **LED branch:** connect 3.3 V through the LED resistor and LED to the same collector node so the transistor sinks the LED current when the alarm is active.

7.2 Breadboard Layout Rules

I kept the AD8232 and ADS1115 close together because the analog output node between them is one of the most sensitive wires in the entire project. On breadboard, this trace should be kept short and should not run in parallel with `SCLK`, `MOSI`, or the buzzer wiring.

The same routing rule was followed throughout the breadboard build: the analog section was kept physically separated from the transistor and buzzer area, the 100 nF capacitors were placed near the device supply pins, the bulk capacitor was kept on the local rail, and the buzzer current return was not allowed to share a narrow daisy-chained path with the analog front-end ground.

8 System Boot, Validation, and Remaining Work

8.1 Boot Sequence

At power-up, the rails stabilize first, then the MCU initializes its clocks and communication peripherals, then the FPGA configures and synchronizes reset release, then the ADS1115 starts its single-shot conversion cycle, and finally the repeating sample pipeline begins.

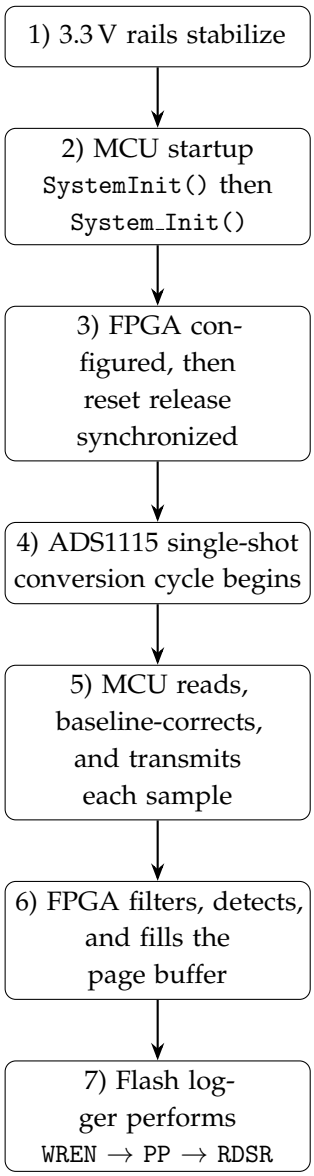

Figure 8.1: Integrated boot and protocol sequence of the final architecture.

8.2 Validation Strategy

The system was validated in stages. First, the MCU-to-ADS1115 I^2C link was verified, because reliable acquisition is a prerequisite for every downstream stage. Second, the MCU-to-FPGA sample transfer was checked. Third, the detector and actuator path were observed through the board LEDs and the external alarm hardware. Finally, the page logger was exercised. This staged approach was not merely convenient; it was the correct way to debug a mixed-signal chain in which each stage depends on the correctness of the previous one.

8.3 Limitations and Future Extensions

The present design is a functional proof of concept rather than a finished instrument. The main limitations are still those expected from a first mixed-signal prototype: breadboard noise sensitivity, a deliberately simple threshold-based QRS detector, the fact that the present QRS threshold is still a bring-up constant rather than a calibrated final value, and the lack of a live telemetry interface for viewing the processed waveform on a host PC.

The next logical extensions are clear. The signal-processing chain can be improved with stronger motion-artifact and baseline-wander suppression. The detector can be made more robust through adaptive or morphology-aware logic. Finally, a UART or other host link can be added so that filtered waveforms, detected beats, and logged pages can be inspected directly rather than inferred only from the actuator behavior.